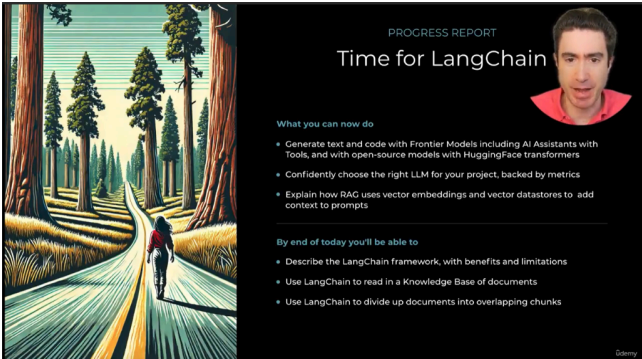




Tiếp tục hành trình với RAG (Retrieval-Augmented Generation):

- Khóa học tiếp tục khám phá RAG, tập trung vào việc sử dụng LangChain để triển khai các ứng dụng RAG hiệu quả hơn.

Ôn lại kiến thức về RAG đã học:

- Nhắc lại "ý tưởng đơn giản" của RAG: trích xuất ngữ cảnh liên quan và đưa vào prompt.
- Nhắc lại khái niệm vector embeddings: biểu diễn văn bản dưới dạng vector thể hiện ý nghĩa của nó.
- Nhắc lại cách sử dụng vector embeddings để chọn ngữ cảnh phù hợp cho prompt một cách thông minh hơn.

Giới thiệu về LangChain:

- Mô tả LangChain framework và các ưu điểm, nhược điểm của nó.
- Sử dụng LangChain để đọc dữ liệu từ cơ sở tri thức (Knowledge Base).
- Sử dụng LangChain để chia dữ liệu thành các đoạn thông tin liên quan (chunks) để lưu trữ trong cơ sở dữ liệu vector và trích xuất sau này.

Mục tiêu bài học hôm nay:

- Hiểu rõ hơn về LangChain và khả năng của nó trong việc xây dựng ứng dụng RAG.
- Áp dụng LangChain để xử lý dữ liệu từ cơ sở tri thức, chuẩn bị cho việc lưu trữ và trích xuất thông tin bằng vector embeddings.

Bối cảnh của LangChain:

- Đoạn trích sẽ cung cấp thông tin về lý do và mục đích ra đời của LangChain.

Giới thiệu về LangChain:

- LangChain là một framework mã nguồn mở được ra mắt vào cuối năm 2022.
- Mục tiêu chính của LangChain là giúp xây dựng các ứng dụng LLM (Large Language Model) một cách nhanh chóng bằng cách kết hợp các chức năng khác nhau thành một chuỗi xử lý.
- LangChain có ngôn ngữ khai báo riêng gọi là LangChain Expression Language (LCEL), nhưng bạn cũng có thể sử dụng Python để làm việc với nó.

Ưu điểm của LangChain:

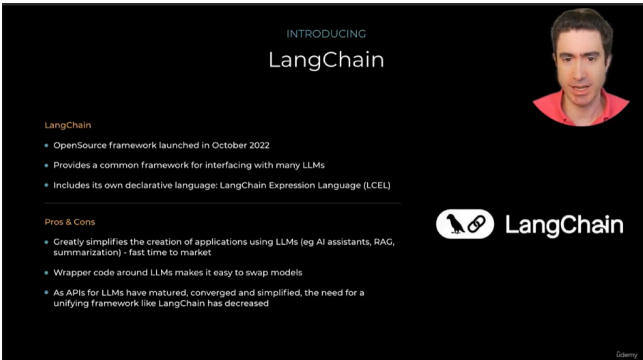
- **Đơn giản hóa việc tạo ứng dụng LLM:** LangChain giúp đơn giản hóa việc xây dựng các ứng dụng như trợ lý ảo và ứng dụng RAG (Retrieval-Augmented Generation) chỉ với một vài dòng code.
- **Tăng tốc độ phát triển:** LangChain giúp giảm thời gian đưa sản phẩm ra thị trường (time to market).
- **Wrapper code tiện lợi:** LangChain cung cấp wrapper code xung quanh các mô hình LLM phổ biến, cho phép bạn dễ dàng thay đổi mô hình mà không cần sửa đổi nhiều code.

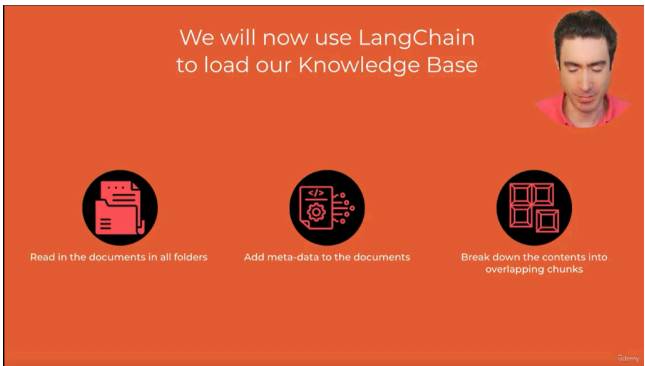
Nhược điểm của LangChain:

- **Giảm nhu cầu theo thời gian:** Khi các API của LLM ngày càng hoàn thiện và trở nên giống nhau hơn, nhu cầu về một framework thống nhất như LangChain đã giảm bớt.
- **Nhiều người tự xây dựng pipeline:** Nhiều nhà phát triển tự xây dựng pipeline riêng cho các ứng dụng như RAG.

Ứng dụng của LangChain trong khóa học:

- Bạn sẽ sử dụng LangChain để tải dữ liệu từ cơ sở tri thức (Knowledge Base).
- Bạn sẽ học cách chia dữ liệu thành các đoạn thông tin liên quan (chunks) để chuẩn bị cho việc lưu trữ và trích xuất bằng vector embeddings.





Đọc tài liệu từ các thư mục:

- Bạn sẽ sử dụng các công cụ của LangChain để đọc tất cả tài liệu được lưu trữ trong các thư mục của cơ sở tri thức.

Thêm metadata vào tài liệu:

- Bạn sẽ thêm metadata (dữ liệu mô tả dữ liệu) vào tài liệu để xác định loại tài liệu.
- Metadata này sẽ hữu ích cho các bước xử lý tiếp theo.

Chia tài liệu thành các đoạn hữu ích (chunks):

- Bạn sẽ sử dụng LangChain để chia tài liệu thành các đoạn văn bản nhỏ hơn, gọi là "chunks".
- Các chunks này được thiết kế để chứa các phần văn bản có liên quan chặt chẽ với nhau.
- Các chunks này sẽ được chuẩn bị để chuyển đổi thành vector embeddings và lưu trữ trong cơ sở dữ liệu vector.

Mục tiêu:

- Chuẩn bị dữ liệu từ cơ sở tri thức để sử dụng trong ứng dụng RAG (Retrieval-Augmented Generation).
- Sử dụng LangChain để tự động hóa các bước xử lý dữ liệu.

Bước tiếp theo:

- Chuyển sang JupyterLab để thực hiện các bước xử lý dữ liệu với LangChain.

```
import os
import glob
from langchain.document_loaders import DirectoryLoader, TextLoader
from langchain.text_splitter import CharacterTextSplitter

# Import for LangChain
from langchain.document_loaders import DirectoryLoader, TextLoader
from langchain.text_splitter import CharacterTextSplitter

# If price is a factor for our company, so we're going to use a low cost model
MODEL = "gpt-4o-mini"
DB_NAME = "vector_db"

# Load environment variables in a file called .env
load_dotenv()
get_openai_api_key() = os.getenv('OPENAI_API_KEY', 'your-key-if-not-using-env')

# Read in documents using LangChain's loaders
# Take everything in all the sub-folders of our knowledgebase
folders = glob.glob("knowledge-base/*")

documents = []
for folder in folders:
    doc_type = os.path.basename(folder)
    loader = DirectoryLoader(folder, glob="**/*.md", loader_cls=TextLoader)
    for doc in loader.load():
        doc.metadata["doc_type"] = doc_type
        documents.append(doc)
```

Môi trường làm việc:

- Tiếp tục sử dụng JupyterLab để thực hành.
- Nhấn mạnh tầm quan trọng của việc tự thực hiện các bước trong notebook để hiểu rõ hơn về các khái niệm.

Nhập các thư viện cần thiết từ LangChain:

- DirectoryLoader và TextLoader: để tải tài liệu từ thư mục và tệp văn bản.
- CharacterTextSplitter: để chia văn bản thành các chunks dựa trên ký tự.

Tải dữ liệu từ cơ sở tri thức sử dụng LangChain:

- Xác định danh sách các thư mục trong cơ sở tri thức (Knowledge Base): "company", "contracts", "employees", "products".
- Sử dụng DirectoryLoader và TextLoader để tải tất cả các tệp văn bản từ các thư mục này.

Thêm metadata vào tài liệu:

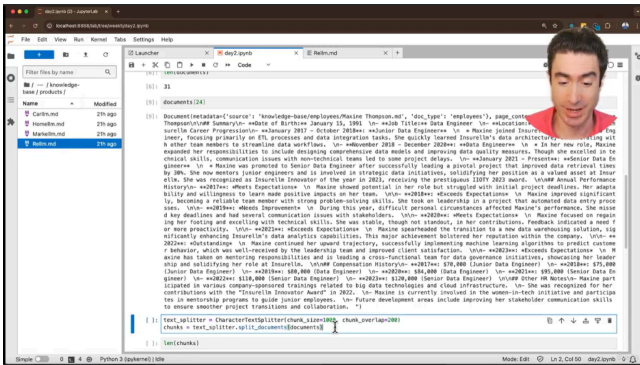
- Thêm metadata doc_type vào mỗi tài liệu để xác định loại tài liệu (company, contracts, employees, products).
- Lưu trữ tất cả tài liệu đã tải và metadata vào một danh sách có tên documents.

Mục tiêu:

- Sử dụng LangChain để tự động hóa quá trình tải dữ liệu từ cơ sở tri thức.
- Chuẩn bị dữ liệu để xử lý tiếp theo (chia thành chunks, tạo vector embeddings).

Lợi ích của LangChain:

- Đơn giản hóa quá trình tải dữ liệu từ nhiều nguồn khác nhau.
- Cung cấp các công cụ tiện lợi để xử lý văn bản.



Kiểm tra kết quả tải tài liệu:

- Sau khi tải tài liệu bằng LangChain, bạn sẽ kiểm tra kết quả để đảm bảo rằng tài liệu đã được tải đúng cách.
- Bạn sẽ thấy rằng 31 đối tượng kiểu document đã được tạo.
- Bạn sẽ kiểm tra nội dung và metadata của một số tài liệu để xác nhận rằng chúng khớp với các tệp gốc.

Sử dụng CharacterTextSplitter để chia tài liệu thành chunks:

- Bạn sẽ sử dụng CharacterTextSplitter từ LangChain để chia mỗi tài liệu thành các chunks (đoạn văn bản nhỏ hơn).
- Bạn sẽ hiểu được hai tham số quan trọng của CharacterTextSplitter:
 - chunk_size: Xác định kích thước tối đa của mỗi chunk (tính bằng ký tự). LangChain sẽ cố gắng chia chunks tại các ranh giới hợp lý (ví dụ: khoảng trắng, dòng trống) để tránh cắt giữa từ hoặc đoạn văn.
 - chunk_overlap: Xác định mức độ trùng lặp giữa các chunks. Điều này giúp đảm bảo rằng các thông tin quan trọng không bị mất khi chia tài liệu.

Lý do sử dụng chunk_overlap:

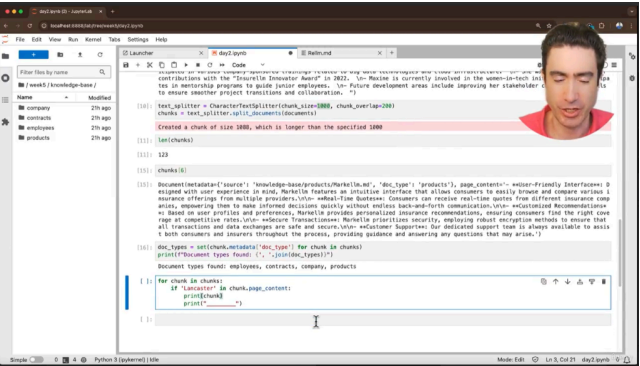
- Ngăn ngừa việc bỏ sót các thông tin quan trọng do từ khóa quan trọng chỉ xuất hiện trong một chunk.
- Tăng khả năng trích xuất các chunks liên quan đến truy vấn của người dùng.

Mục tiêu:

- Chia tài liệu thành các chunks nhỏ hơn để cải thiện hiệu suất tìm kiếm thông tin.
- Sử dụng CharacterTextSplitter của LangChain để tự động hóa quá trình chia văn bản.

Bước tiếp theo:

- Thực thi code để chia tài liệu thành chunks.



Kiểm tra kết quả chia chunks:

- Bạn sẽ thấy rằng quá trình chia văn bản thành chunks đã tạo ra 123 chunks từ 31 tài liệu gốc.
- Bạn sẽ kiểm tra nội dung và metadata của một số chunks để hiểu rõ hơn về cách LangChain chia văn bản.
- Bạn sẽ thấy rằng LangChain cố gắng chia chunks tại các ranh giới hợp lý để đảm bảo rằng các chunks có ý nghĩa.

Kiểm tra metadata của chunks:

- Bạn sẽ kiểm tra metadata doc_type của các chunks để đảm bảo rằng chúng phản ánh đúng loại tài liệu gốc.
- Bạn sẽ thấy rằng tất cả bốn loại tài liệu (employees, contracts, company, products) đều được đại diện trong các chunks.

Tìm kiếm chunks dựa trên từ khóa:

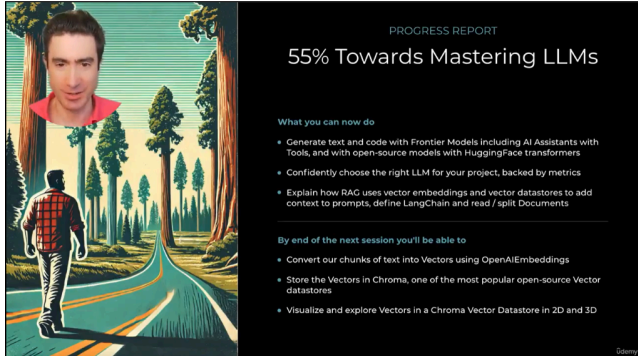
- Bạn sẽ thực hiện tìm kiếm đơn giản dựa trên từ khóa (ví dụ: "Lancaster", "Avery", "CEO") để minh họa hạn chế của phương pháp tìm kiếm dựa trên văn bản thuần túy.
- Bạn sẽ thấy rằng tìm kiếm dựa trên từ khóa có thể bỏ sót các thông tin quan trọng nếu từ khóa không xuất hiện trong văn bản.

Giới thiệu về tìm kiếm vector (vector search):

- Bạn sẽ được giới thiệu về tìm kiếm vector, một phương pháp tìm kiếm thông minh hơn dựa trên ý nghĩa của văn bản thay vì chỉ từ khóa.
- Tìm kiếm vector có thể tìm thấy các chunks liên quan ngay cả khi chúng không chứa từ khóa cụ thể.

Bước tiếp theo:

- Bạn sẽ được khuyến khích tự thực hành với các chunks để hiểu rõ hơn về cách chúng được tạo ra.
- Bạn sẽ chuẩn bị cho bước tiếp theo: chuyển đổi chunks thành vector embeddings và lưu trữ chúng trong cơ sở dữ liệu vector.



Tiến độ học tập:

- Đã hoàn thành 55% khóa học về làm chủ LLMs (Large Language Models).
- Đã đạt được tiến bộ đáng kể trong việc tìm hiểu về RAG (Retrieval-Augmented Generation).

Kiến thức đã đạt được:

- Bạn đã có kiến thức nền tảng về lý do sử dụng vectors trong RAG.
- Bạn đã học cách sử dụng LangChain để tải tài liệu, chia chúng thành chunks và thêm metadata.
- Bạn đã thực hành và hiểu rõ cách LangChain hoạt động.

Nội dung học tập tiếp theo:

- Học cách chuyển đổi các chunks văn bản thành vector embeddings sử dụng OpenAIEmbeddings.
- Học cách lưu trữ các vector embeddings trong Chroma, một cơ sở dữ liệu vector mã nguồn mở phổ biến.
- Học cách trực quan hóa và khám phá các vector embeddings trong Chroma ở dạng 2D và 3D.

Mục tiêu:

- Hiểu rõ hơn về cách vector embeddings được sử dụng trong RAG.
- Nắm vững cách lưu trữ và truy xuất vector embeddings từ cơ sở dữ liệu vector.
- Trực quan hóa vector embeddings để hiểu rõ hơn về ý nghĩa của chúng.